

Praktikumsbericht – Projekt

Canny Edge Detection auf der GPU

Matthias Stefan

Thomas Jurczyk

24. Juni 2026

Inhaltsverzeichnis

1	Voraussetzungen und Projektaufbau	2
1.1	Hardware-Spezifikationen	2
1.2	Projektstruktur	2
1.3	Build-System und Kompilierung	2
2	Algorithmus	3
2.1	Überblick	3
2.2	Details	3
2.2.1	Schritt 1: Gaussian Blur	3
2.2.2	Schritt 2: Gradient Berechnung (Sobel)	3
2.2.3	Schritt 3: Non-Maximum Suppression	3
2.2.4	Schritt 4: Hysteresis Thresholding	4
3	Implementierung	5
3.1	Naive Implementierung	5
3.1.1	Schritt 1: Gaussian Blur	5
3.1.2	Shared Memory: Gaussian Blur	6
3.1.3	Schritt 2: Gradient Berechnung (Sobel)	6
3.1.4	Schritt 3: Non-Maximum Suppression	6
3.1.5	Schritt 4: Hysteresis Thresholding	8
3.2	Optimierungen	8
4	Ergebnisse und Analyse	9
4.1	Korrektheit	9
4.2	Performance-Messungen	9
4.3	Analyse	9
5	Fazit	10

1 Voraussetzungen und Projektaufbau

1.1 Hardware-Spezifikationen

System: Matthias Stefan

System: Thomas Jurczyk

1.2 Projektstruktur

1.3 Build-System und Kompilierung

2 Algorithmus

2.1 Überblick

Der Canny-Algorithmus ist ein mehrstufiges Verfahren zur Kantendetektion in Graustufenbildern. Canny definiert drei Kriterien für einen optimalen Kantendetektor: gute Detektion, präzise Lokalisierung sowie exakt eine Antwort pro Kante – jede Kante soll auf eine Breite von einem Pixel supprimiert werden [1, S. 680]. Die Verarbeitung erfolgt in vier aufeinanderfolgenden Schritten, wobei der Output jedes Schritts als Input des nächsten dient (siehe Abbildung 1). Die einzelnen Schritte werden in den kommenden Abschnitten näher erläutert.

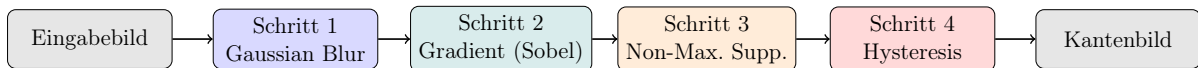


Abbildung 1: Canny Edge Detection Pipeline

2.2 Details

2.2.1 Schritt 1: Gaussian Blur

Um einen mathematisch optimalen Kantendetektor herzuleiten, trifft Canny eine Annahme über das Rauschen im Bild: „*We will assume that the image consists of the edge and additive white Gaussian noise*“ [1, S. 680]. Da dieses Rauschen fälschlicherweise als Kante detektiert werden kann, muss das Bild zunächst geglättet werden. Canny zeigt, dass der optimale Filter für dieses Rauschmodell durch die erste Ableitung einer Gaußfunktion approximiert werden kann [1, S. 687]:

$$G(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \quad (1)$$

Der Parameter σ kontrolliert die Stärke der Glättung. Zwischen Rauschunterdrückung und Lokalisierungsgenauigkeit besteht jedoch ein grundlegender Zielkonflikt – Canny bezeichnet dies als „*uncertainty principle*“ zwischen Detektion und Lokalisierung [1, S. 684].

2.2.2 Schritt 2: Gradient Berechnung (Sobel)

2.2.3 Schritt 3: Non-Maximum Suppression

Schritt 2 liefert für jeden Pixel einen Gradientenbetrag, der die lokale Kantenstärke beschreibt, die resultierenden Kanten sind dabei an vielen Stellen breiter als ein Pixel. Canny fordert jedoch präzise Lokalisierung: „*The points marked as edge points by the operator should be as close as possible to the center of the true edge*“ [1, S. 680]. Non-Maximum Suppression reduziert diese breite Gradientenantwort auf eine Breite von einem Pixel, indem entlang der Gradientenrichtung nur das lokale Maximum erhalten bleibt.

Formal lässt sich der gesuchte Kantenpunkt als Nullstelle der zweiten Richtungsableitung des geglätteten Bildes beschreiben:

$$\frac{\partial^2}{\partial n^2}(G * I) = 0 \quad (2)$$

wobei n die Richtung des Gradienten bezeichnet und $G * I$ das geglättete Bild [1, S. 691]. Die Nullstelle der zweiten Ableitung markiert exakt den Punkt, an dem die erste Ableitung, die Gradientenstärke entlang n , ihr lokales Maximum besitzt. Da die Kantenrichtung n nicht vorab bekannt ist, wird sie aus der geglätteten Gradientenrichtung geschätzt [1, Gl. 46, S. 691]:

$$n = \frac{\nabla(G * I)}{|\nabla(G * I)|} \quad (3)$$

In der diskreten Implementierung wird Gleichung 2 approximiert, indem die Gradientenmagnitude $|\nabla(G * I)|$ jedes Pixels mit den beiden direkten Nachbarn entlang der auf vier Klassen ($0^\circ/45^\circ/90^\circ/135^\circ$) quantisierten Gradientenrichtung verglichen wird. Ein Pixel überlebt die Suppression genau dann, wenn seine Magnitude größer oder gleich beiden Nachbarn ist, andernfalls wird er auf null gesetzt.

2.2.4 Schritt 4: Hysteresis Thresholding

3 Implementierung

3.1 Naive Implementierung

3.1.1 Schritt 1: Gaussian Blur

Die Gaußgewichte werden zunächst auf der CPU als zweidimensionales Raster der Größe $(2 \cdot \text{blur_radius} + 1)$ vorberechnet und zwischengespeichert – jeder Eintrag enthält das normalisierte Gaußgewicht für den entsprechenden Versatz (i, j) relativ zum Zielpixel (Gleichung 1). Die Funktion `calculate_gaussian_weights()` erhält dazu als Parameter `blur_radius` sowie `sigma` zur Steuerung der Glättungsstärke. Nach der Auswertung werden die Gewichte normalisiert, sodass ihre Summe 1 ergibt und die Helligkeit des Bildes erhalten bleibt. Das Gewichtsraster wird einmalig vor dem Kernelaufwurf auf die GPU übertragen und steht dort allen Threads als gemeinsam genutzter Read-only-Buffer zur Verfügung – so wird vermieden, dass die Gewichte für jeden Pixel neu berechnet werden müssen. Das resultierende 3×3 Gewichtsraster ist in Abbildung 2 links in Rot dargestellt.

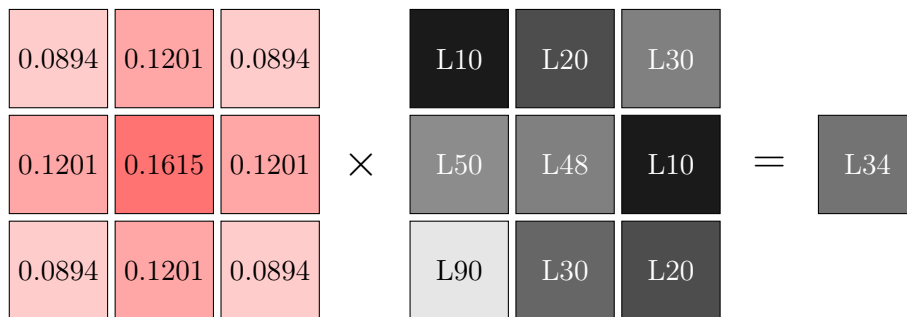


Abbildung 2: Gaussian Blur – gewichteter Mittelwert am Beispielpixel $L48 \rightarrow L34$

Die eigentliche Filterung findet im CUDA-Kernel `gaussian_filter()` statt. Jeder Thread ist für genau einen Ausgabepixel zuständig – die Thread-Position im Bild ergibt sich aus `blockIdx` und `threadIdx` (Listing 1, Zeile 1–2). Threads außerhalb des Bildrandes werden sofort verworfen (Zeile 3). Anschließend iteriert jeder Thread über das Gewichtsraster und akkumuliert den gewichteten Mittelwert der Nachbarpixel (Zeile 5–10). Pixel außerhalb des Bildrandes werden dabei auf 0 gesetzt (Zero-Padding). Das Ergebnis wird abschließend in den Ausgabepuffer geschrieben (Zeile 12), wie in Abbildung 2 rechts dargestellt.

```

1 x = blockIdx.x * blockDim.x + threadIdx.x
2 y = blockIdx.y * blockDim.y + threadIdx.y
3 if (x, y) ausserhalb des Bildes: return
4
5 color = 0.0
6 for j = -radius to +radius:
7     for i = -radius to +radius:
8         if (x+i, y+j) innerhalb des Bildes:
9             pixel = src_buffer[(y+j) * width + (x+i)]
10            color += pixel * weights[(j+r) * blur_size + (i+r)]
11
12 out_buffer[y * width + x] = color

```

Listing 1: Pseudocode – `gaussian_filter` Kernel

In der naiven Implementierung greift jeder Thread dabei wiederholt auf den globalen GPU-Speicher zu. Da benachbarte Threads überlappende Pixelbereiche lesen, werden dieselben Pixel mehrfach aus dem langsamen DRAM geladen – dies motiviert die spätere Optimierung mit Shared Memory (siehe Abschnitt 3.1.2).

Das Ergebnis der Filterung ist in Abbildung 3 dargestellt.



Abbildung 3: Vergleich Original (links) vs. Gaussian Blur (rechts, $\sigma = 25$, radius = 12). Die Parameter wurden bewusst extrem gewählt, um den Unterschied deutlicher sichtbar zu machen – in den weiteren Verarbeitungsschritten wird ein deutlich sanfterer Filter verwendet.

3.1.2 Shared Memory: Gaussian Blur

3.1.3 Schritt 2: Gradient Berechnung (Sobel)

3.1.4 Schritt 3: Non-Maximum Suppression

Die Non-Maximum Suppression wird im CUDA-Kernel `non_maximum_suppression()` umgesetzt. Jeder Thread ist dabei für genau einen Pixel zuständig, die Thread-Position ergibt sich wie in Schritt 1 und 2 aus `blockIdx` und `threadIdx`. Als Input erhält der Kernel den Gradientenbetrag (`magnitude_buffer`) sowie die Gradientenrichtung in Radiant (`direction_buffer`) aus Schritt 2.

Zunächst wird die Gradientenrichtung von Radiant in Grad umgerechnet und auf den Bereich $[0^\circ, 180^\circ)$ gefaltet, da eine Kante keine Orientierung besitzt, eine Kante bei 170° und eine bei -10° beschreiben dieselbe Achse. Anschließend wird die Richtung auf eine von vier Klassen quantisiert, die den vier möglichen Achsen im diskreten Pixelraster entsprechen (siehe Abbildung 4).

Je nach Richtungsklasse werden die Offsets (dx, dy) zum Nachbarpixel bestimmt (Listing 2, Zeile 18–22). Der Pixel überlebt die Suppression genau dann, wenn seine Magnitude größer oder gleich beiden Nachbarn entlang dieser Achse ist (Zeile 24–26), andernfalls wird er auf null gesetzt:

```
1  __global__ void non_maximum_suppression(  
2      const uint8_t* magnitude_buffer,  
3      const float* direction_buffer,  
4      const int32_t width,  
5      const int32_t height,  
6      uint8_t* out_nms_buffer)  
7  {  
8      const auto x = static_cast<int32_t>(blockIdx.x * blockDim.x + threadIdx.x);  
9      const auto y = static_cast<int32_t>(blockIdx.y * blockDim.y + threadIdx.y);  
10     if (x >= width || y >= height)  
11         return;  
12 }
```

```

13  const uint8_t magnitude = magnitude_buffer[y * width + x];
14  const float direction = direction_buffer[y * width + x];
15
16  float deg = direction * (180.0f / CUDART_PI_F);
17  deg = fmodf(deg + 180.0f, 180.0f);
18
19  int32_t dx, dy;
20  if (deg < 22.5f) { dx = 1; dy = 0; }
21  else if (deg < 67.5f) { dx = 1; dy = -1; }
22  else if (deg < 112.5f) { dx = 0; dy = 1; }
23  else if (deg < 157.5f) { dx = 1; dy = 1; }
24  else { dx = 1; dy = 0; }
25
26  const uint8_t neighbor_a =
27      sample_clamped(magnitude_buffer, width, height, x + dx, y + dy);
28  const uint8_t neighbor_b =
29      sample_clamped(magnitude_buffer, width, height, x - dx, y - dy);
30
31  out_nms_buffer[y * width + x] =
32      (magnitude >= neighbor_a && magnitude >= neighbor_b) ? magnitude : 0;
33  }

```

Listing 2: non_maximum_suppression Kernel

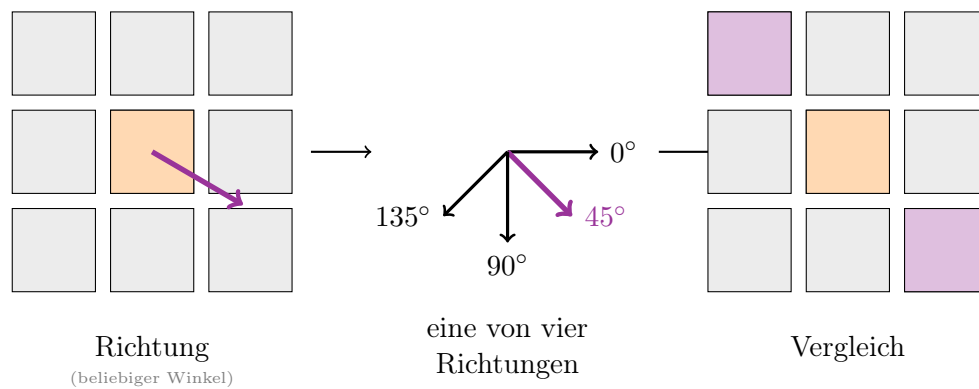


Abbildung 4: Non-Maximum Suppression: Die Gradientenrichtung wird auf eine von vier Achsen quantisiert (0° , 45° , 90° , 135°). Anschließend wird der Pixel mit den beiden farbig markierten Nachbarn entlang der gewählten Achse verglichen.

Das Ergebnis der Non-Maximum Suppression ist in Abbildung 5 dargestellt. Im Vergleich zur Gradientenmagnitude aus Schritt 2 sind die Kanten auf eine Breite von einem Pixel reduziert.



Abbildung 5: Vergleich Gradientenmagnitude (links) vs. Non-Maximum Suppression (rechts), die breite Gradientenantwort wird auf eine Breite von einem Pixel reduziert. (Placeholder – wird ersetzt)

3.1.5 Schritt 4: Hysteresis Thresholding

3.2 Optimierungen

4 Ergebnisse und Analyse

4.1 Korrektheit

4.2 Performance-Messungen

4.3 Analyse

5 Fazit

Literatur

- [1] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, November 1986.